2.1 Cart-Pole Swing-Up with Limited Actuation

2.1(a)

We want the constraint $s_{k+1} = f_{\mathrm{d}}(s_k, u_k)$ to have the form $s_{k+1} = A_k s_k + B_k u_k + c_k$ to derive the convex approximation of the OCP at iteration i, around $(s_k^{(i)}, u_k^{(i)})$.

The linearization of $f_{\mathrm{d}}(s_k, u_k)$ becomes:

$$f_{\mathrm{d}}(s_k^{(i)}, u_k^{(i)}) = f_(s_k^{(i)}, u_k^{(i)}) + \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)})(s_k - s^{(i)}) + \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)})(u_k - u^{(i)})$$

After expanding terms, we come up with

$$A_k = \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)}), \qquad B_k = \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)}),$$

$$c_k = f_d(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}.$$

The convex subproblem is

$$\begin{aligned}
\min_{s,u} \quad & \sum_{k=0}^{N-1} \left((s_k - s_{\mathrm{goal}})^T Q (s_k - s_{\mathrm{goal}}) + u_k^T R u_k\right) + (s_N - s_{\mathrm{goal}})^T P (s_N - s_{\mathrm{goal}}) \\
\text{s.t.} \quad & s_0 = \bar{s}_0, \\
& s_{k+1} = A_k s_k + B_k u_k + c_k, \\
& -u_{\max} \le u_k \le u_{\max}, \\
& \|s_k - s_k^{(i)}\|_\infty \le \rho, \qquad \|u_k - u_k^{(i)}\|_\infty \le \rho.
\end{aligned}$$

2.1(b)

```
A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
c = f(s, u) - A @ s - B @ u
```

2.1(d)

First use SCP to compute a feasible swing-up trajectory. Once the pendulum is near $(0, \pi, 0, 0)$, switch to a local LQR controller designed around $(0, \pi, 0, 0)$ for stabilization.

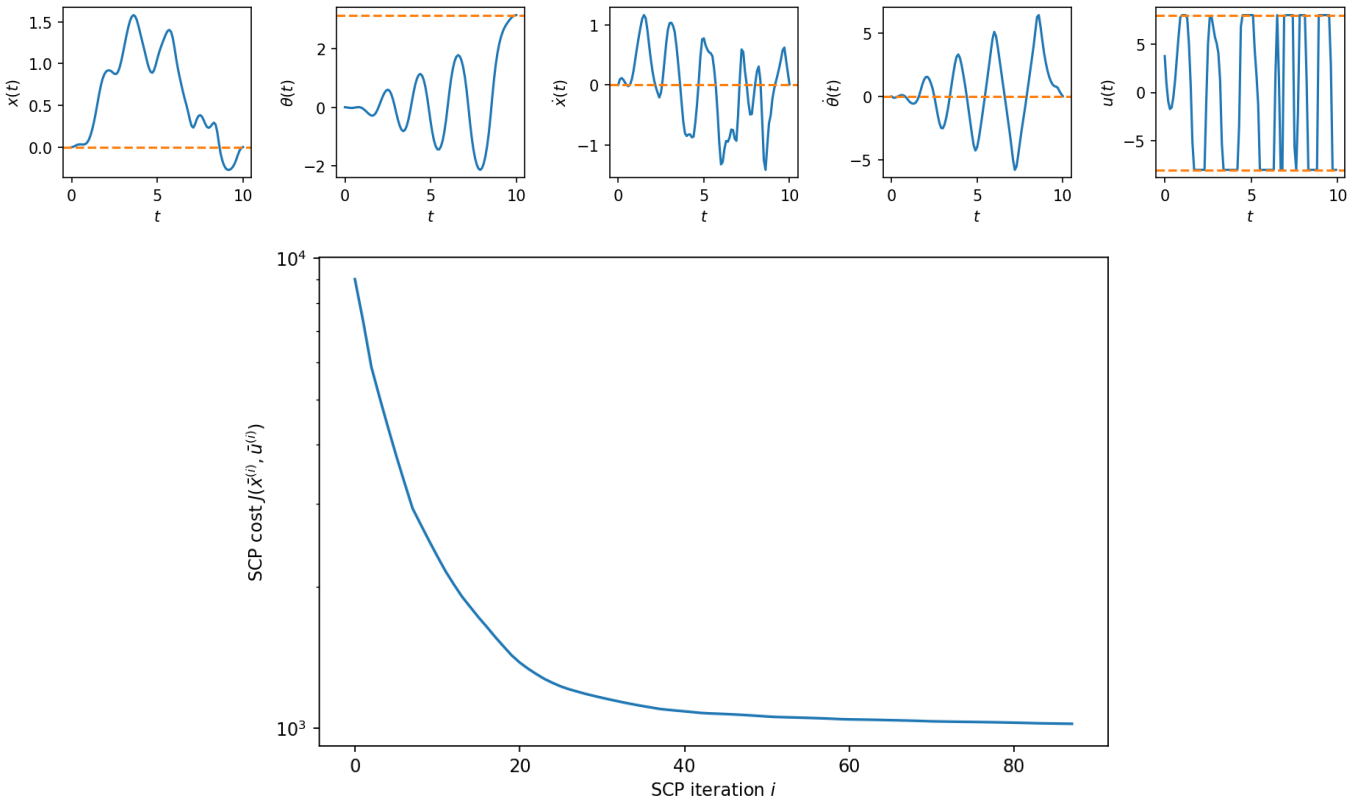


Figure 1: SCP swing-up trajectory with bounded input and trust-region updates.

Code

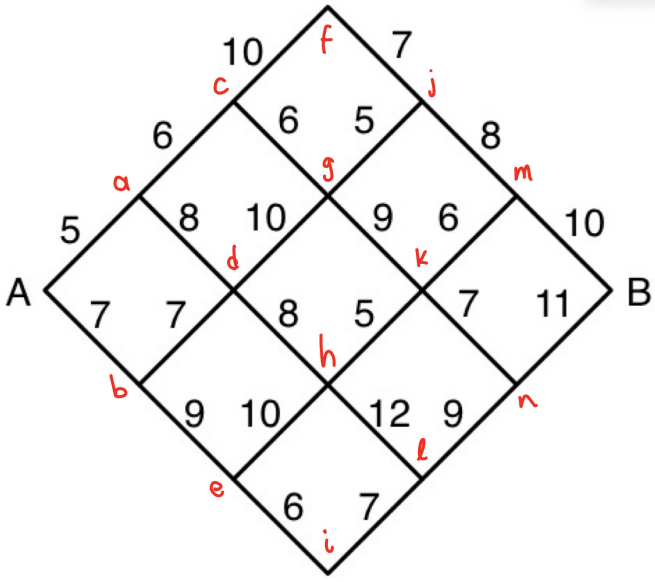
Code in Appendix: `cartpole_swingup_constrained.py`.

2.2 Shortest Path Through a Grid

2.2(a)

Following DP, the next table, based on Fig. , with each node cost is computed.

node	J(node)
B	0
m	10
n	11
j	18
k	16
l	20
f	25
g	23
h	21
i	27
c	29
d	29
e	31
a	35
b	36
A	40



The shortest path has cost $\boxed{40}$. And the optimal path is:

$$A \to a \to c \to g \to j \to m \to B.$$

2.2(b)

For an n-segment grid, each route consists of n up moves and n down moves, so exhaustive search evaluates $\boxed{\binom{2n}{n}}$ routes.

Dynamic programming evaluates each nonterminal node. The number of DP evaluations is $\boxed{n(n+2)}$.

For $n = 3$, these give $\binom{6}{3} = 20$ and $3(5) = 15$.

2.3 Cart-Pole Balance

2.3(a)

With $\tilde{s}_k = s_k - \bar{s}$, Euler discretization gives

$$\tilde{s}_{k+1} \approx A\tilde{s}_k + Bu_k$$

where

$$A = I + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c+m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \qquad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}$$

2.3(b)

Using Riccati recursion until $\|P_k - P_{k-1}\|_{\max} < 10^{-4}$ gives

```
K: [[   0.73  -231.85    4.22  -68.24]]
```

2.3(c)

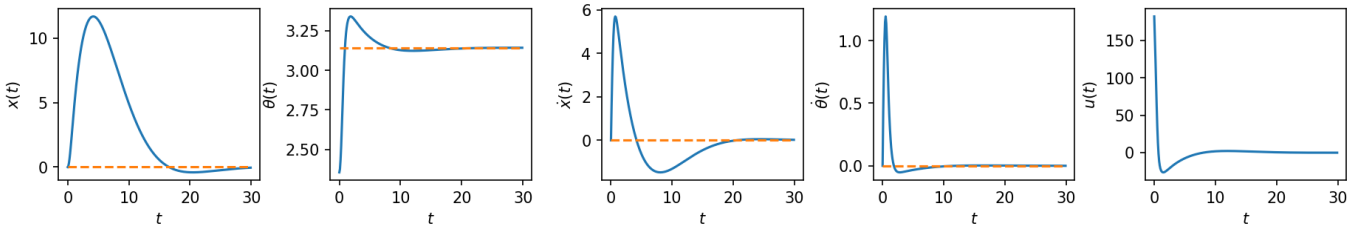


Figure 2: Infinite-horizon LQR controller for cat-pole balance

2.3(d)

i. We can reuse A and B because the dynamics Jacobians along the reference do not depend on x, and the desired trajectory keeps $\theta = \pi$, $\dot{\theta} = 0$. Then the same upright linearization is valid along the reference.

ii.

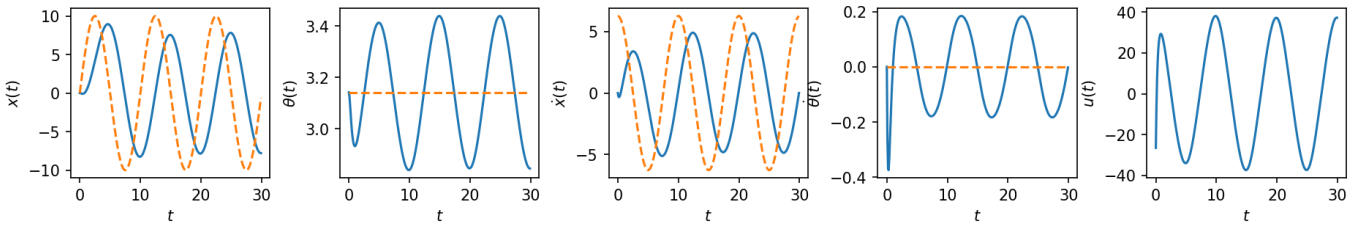


Figure 3: Tracking new reference trajectory $s(t)$ with LQR feedback.

iii. The pendulum must tilt to generate the horizontal cart acceleration from the reference, so keeping $\theta(t) \equiv \pi$ while moving the cart sinusoidally is not a dynamically consistent upright trajectory without feedforward compensation.

2.4 Cart-Pole Swing-Up

2.4(a)

The requested one-line JAX linearization is

```
A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
```

2.4(b)

Let

$$\tilde{s}_k = s_k - \bar{s}_k, \qquad \tilde{u}_k = u_k - \bar{u}_k.$$

Expanding the quadratic cost gives linear terms

$$q_N = Q_N(\bar{s}_N - s_{\text{goal}}), \qquad q_k = Q(\bar{s}_k - s_{\text{goal}}), \qquad r_k = R\bar{u}_k.$$

2.4(c)

The iLQR implementation is in `code/cartpole_swingup.py`. It computes linearizations, performs the Riccati backward pass for feedback gains Y_k and offsets y_k, updates the nominal trajectory, and uses backtracking to keep the update stable.

2.4(d)

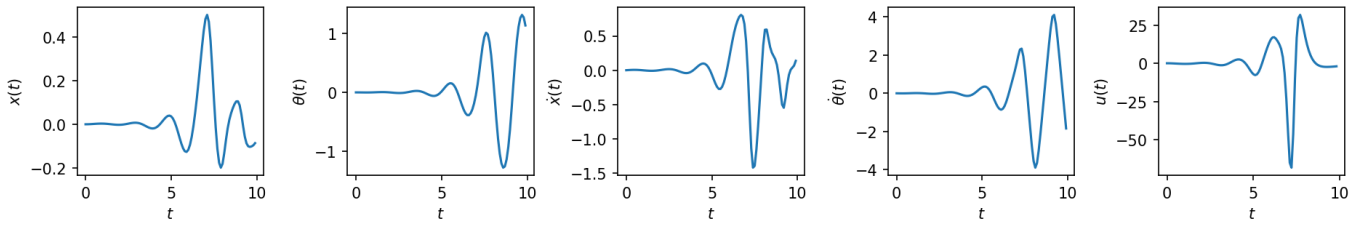


Figure 4: Open-loop application of the iLQR control sequence.

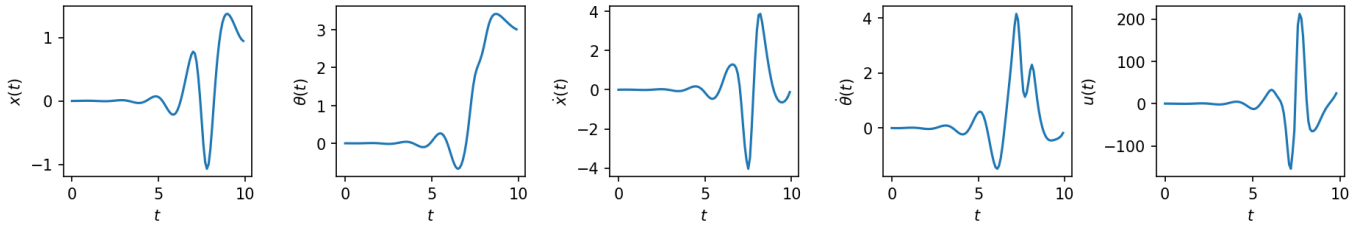


Figure 5: Closed-loop iLQR policy.

2.5 Machine Maintenance

Let $V_R(h)$ and $V_B(h)$ denote the optimal expected profit with h weeks remaining when the machine starts the week running or broken.

$$\begin{aligned} V_R(h) &= \max\{40 + 0.6V_R(h-1) + 0.4V_B(h-1), \\ &\qquad 30 + 0.3V_R(h-1) + 0.7V_B(h-1)\}, \\ V_B(h) &= \max\{20 + 0.6V_R(h-1) + 0.4V_B(h-1),\ -50 + V_R(h-1)\}. \end{aligned}$$

With $V_R(0) = V_B(0) = 0$, the computed values are

h	0	1	2	3
$V_R(h)$	0	40	72	104
$V_B(h)$	0	20	52	84

The optimal policy is to perform preventive maintenance when running and to repair when broken. Since the machine is new at week 1 and guaranteed to run through that week, the total expected profit over four weeks is

$$\boxed{100 + V_R(3) = 204}.$$

Code Appendix

Homework part	Code file
2.1(b), 2.1(c), 2.1(d)	`cartpole_swingup_constrained.py`
2.3(a), 2.3(b), 2.3(c), 2.3(d)	`cartpole_balance.py`
2.4(a), 2.4(c), 2.4(d)	`cartpole_swingup.py`

cartpole_swingup_constrained.py

Answers 2.1(b), 2.1(c), and 2.1(d).

```
"""
Starter code for the problem "Cart-pole swing-up with limited actuation".

Autonomous Systems Lab (ASL), Stanford University
"""

from functools import partial
from pathlib import Path

from animations import animate_cartpole

import cvxpy as cvx

import jax
import jax.numpy as jnp

import matplotlib.pyplot as plt

import numpy as np

from tqdm import tqdm

FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

@partial(jax.jit, static_argnums=(0,))
@partial(jax.vmap, in_axes=(None, 0, 0))
def affinize(f, s, u):
    """Affinize the function `f(s, u)` around `(s, u)`.

    Arguments
    ---------
    f : callable
        A nonlinear function with call signature `f(s, u)`.
    s : numpy.ndarray
        The state (1-D).
    u : numpy.ndarray
        The control input (1-D).

    Returns
    -------
    A : jax.numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `s`.
    B : jax.numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `u`.
    c : jax.numpy.ndarray
        The offset term in the first-order Taylor expansion of `f` at `(s, u)`
        that sums all vector terms strictly dependent on the nominal point
        `(s, u)` alone.
```

```
    """
        # PART (b) ##############################################################
        # INSTRUCTIONS: Use JAX to affinize `f` around `(s, u)` in two lines.
        A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
        c = f(s, u) - A @ s - B @ u
        # END PART (b) ##########################################################
        return A, B, c


def solve_swingup_scp(f, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters):
    """Solve the cart-pole swing-up problem via SCP.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
    s_goal : numpy.ndarray
        The goal state (1-D).
    N : int
        The time horizon of the LQR cost function.
    P : numpy.ndarray
        The terminal state cost matrix (2-D).
    Q : numpy.ndarray
        The state stage cost matrix (2-D).
    R : numpy.ndarray
        The control stage cost matrix (2-D).
    u_max : float
        The bound defining the control set `[-u_max, u_max]`.
    ρ : float
        Trust region radius.
    eps : float
        Termination threshold for SCP.
    max_iters : int
        Maximum number of SCP iterations.

    Returns
    -------
    s : numpy.ndarray
        A 2-D array where `s[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u : numpy.ndarray
        A 2-D array where `u[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    J : numpy.ndarray
        A 1-D array where `J[i]` is the SCP sub-problem cost after the i-th
        iteration, for `i = 0, 1, ..., (iteration when convergence occured)`
    """
    n = Q.shape[0]  # state dimension
    m = R.shape[0]  # control dimension

    # Initialize dynamically feasible nominal trajectories
    u = np.zeros((N, m))
    s = np.zeros((N + 1, n))
    s[0] = s0
    for k in range(N):
        s[k + 1] = fd(s[k], u[k])

    # Do SCP until convergence or maximum number of iterations is reached
    converged = False
    J = np.zeros(max_iters + 1)
```

```
    J[0] = np.inf
    for i in (prog_bar := tqdm(range(max_iters))):
        s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, ρ)
        dJ = np.abs(J[i + 1] - J[i])
        prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
        if dJ < eps:
            converged = True
            print("SCP converged after {} iterations.".format(i))
            break
    if not converged:
        raise RuntimeError("SCP did not converge!")
    J = J[1 : i + 1]
    return s, u, J


def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, ρ):
    """Solve a single SCP sub-problem for the cart-pole swing-up problem.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
    s_goal : numpy.ndarray
        The goal state (1-D).
    s_prev : numpy.ndarray
        The state trajectory around which the problem is convexified (2-D).
    u_prev : numpy.ndarray
        The control trajectory around which the problem is convexified (2-D).
    N : int
        The time horizon of the LQR cost function.
    P : numpy.ndarray
        The terminal state cost matrix (2-D).
    Q : numpy.ndarray
        The state stage cost matrix (2-D).
    R : numpy.ndarray
        The control stage cost matrix (2-D).
    u_max : float
        The bound defining the control set `[-u_max, u_max]`.
    ρ : float
        Trust region radius.

    Returns
    -------
    s : numpy.ndarray
        A 2-D array where `s[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u : numpy.ndarray
        A 2-D array where `u[k]` is the open-loop state at time step `k`,
        for `k = 0, 1, ..., N-1`
    J : float
        The SCP sub-problem cost.
    """
    A, B, c = affinize(f, s_prev[:-1], u_prev)
    A, B, c = np.array(A), np.array(B), np.array(c)
    n = Q.shape[0]
    m = R.shape[0]
    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    # PART (c) ############################################################
```

```
    # INSTRUCTIONS: Construct the convex SCP sub-problem.
    objective = cvx.quad_form(s_cvx[N] - s_goal, P)
    constraints = [s_cvx[0] == s0]
    for k in range(N):
        objective += cvx.quad_form(s_cvx[k] - s_goal, Q)
        objective += cvx.quad_form(u_cvx[k], R)
        constraints += [
            s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k],
            cvx.abs(u_cvx[k, 0]) <= u_max,
            cvx.norm_inf(s_cvx[k] - s_prev[k]) <= ρ,
            cvx.norm_inf(u_cvx[k] - u_prev[k]) <= ρ,
        ]
    constraints.append(cvx.norm_inf(s_cvx[N] - s_prev[N]) <= ρ)
    # END PART (c) ############################################################

    prob = cvx.Problem(cvx.Minimize(objective), constraints)
    prob.solve()
    if prob.status != "optimal":
        raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
    s = s_cvx.value
    u = u_cvx.value
    J = prob.objective.value
    return s, u, J


def cartpole(s, u):
    """Compute the cart-pole state derivative."""
    mp = 1.0  # pendulum mass
    mc = 4.0  # cart mass
    L = 1.0  # pendulum length
    g = 9.81  # gravitational acceleration

    x, θ, dx, dθ = s
    sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = jnp.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


def discretize(f, dt):
    """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""

    def integrator(s, u, dt=dt):
        k1 = dt * f(s, u)
        k2 = dt * f(s + k1 / 2, u)
        k3 = dt * f(s + k2 / 2, u)
        k4 = dt * f(s + k3, u)
        return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    return integrator


# Define constants
n = 4  # state dimension
m = 1  # control dimension
```

```python
s_goal = np.array([0, np.pi, 0, 0])  # desired upright pendulum state
s0 = np.array([0, 0, 0, 0])  # initial downright pendulum state
dt = 0.1  # discrete time resolution
T = 10.0  # total simulation time
P = 1e3 * np.eye(n)  # terminal state cost matrix
Q = np.diag([1e-2, 1.0, 1e-3, 1e-3])  # state cost matrix
R = 1e-3 * np.eye(m)  # control cost matrix
ρ = 1.0  # trust region parameter
u_max = 8.0  # control effort bound
eps = 5e-1  # convergence tolerance
max_iters = 100  # maximum number of SCP iterations
animate = True  # flag for animation

# Initialize the discrete-time dynamics
fd = jax.jit(discretize(cartpole, dt))

# Solve the swing-up problem with SCP
t = np.arange(0.0, T + dt, dt)
N = t.size - 1
s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, ρ, eps, max_iters)

# Simulate open-loop control
for k in range(N):
    s[k + 1] = fd(s[k], u[k])

# Plot state and control trajectories
fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
plt.subplots_adjust(wspace=0.45)
labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
labels_u = (r"$u(t)$",)
for i in range(n):
    ax[i].plot(t, s[:, i])
    ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
    ax[i].set_xlabel(r"$t$")
    ax[i].set_ylabel(labels_s[i])
for i in range(m):
    ax[n + i].plot(t[:-1], u[:, i])
    ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
    ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
    ax[n + i].set_xlabel(r"$t$")
    ax[n + i].set_ylabel(labels_u[i])
plt.savefig(FIG_DIR / "cartpole_swingup_constrained.png", bbox_inches="tight")

# Plot cost history over SCP iterations
fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
ax.semilogy(J)
ax.set_xlabel(r"SCP␣iteration␣$i$")
ax.set_ylabel(r"SCP␣cost␣$J(\bar{x}^{(i)},␣\bar{u}^{(i)})$")
plt.savefig(FIG_DIR / "cartpole_swingup_constrained_cost.png", bbox_inches="tight")
# plt.show()

# Animate the solution
if animate:
    fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
    ani.save(FIG_DIR / "cartpole_swingup_constrained.mp4", writer="ffmpeg")
    # plt.show()
```

cartpole_balance.py

Answers 2.3(a), 2.3(b), 2.3(c), and 2.3(d).

```python
"""
Solution code for the problem "Cart-pole balance".

Autonomous Systems Lab (ASL), Stanford University
"""

import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt
from pathlib import Path

from animations import animate_cartpole

# Constants
n = 4  # state dimension
m = 1  # control dimension
mp = 2.0  # pendulum mass
mc = 10.0  # cart mass
L = 1.0  # pendulum length
g = 9.81  # gravitational acceleration
dt = 0.1  # discretization time step
animate = True  # whether or not to animate results

FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

def cartpole(s: np.ndarray, u: np.ndarray) -> np.ndarray:
    """Compute the cart-pole state derivative

    Args:
        s (np.ndarray): The cartpole state: [x, theta, x_dot, theta_dot], shape (n,)
        u (np.ndarray): The cartpole control: [F_x], shape (m,)

    Returns:
        np.ndarray: The state derivative, shape (n,)
    """
    x, θ, dx, dθ = s
    sinθ, cosθ = np.sin(θ), np.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = np.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


def reference(t: float) -> np.ndarray:
    """Compute the reference state (s_bar) at time t

    Args:
        t (float): Evaluation time

    Returns:
        np.ndarray: Reference state, shape (n,)
    """
    a = 10.0  # Amplitude
    T = 10.0  # Period
```

```
63
64      # PART (d) ###################################################
65      # INSTRUCTIONS: Compute the reference state for a given time
66      omega = 2 * np.pi / T
67      return np.array([a * np.sin(omega * t), np.pi, a * omega * np.cos(omega * t), 0.0])
68      # END PART (d) #################################################
69
70
71  def ricatti_recursion(
72      A: np.ndarray, B: np.ndarray, Q: np.ndarray, R: np.ndarray
73  ) -> np.ndarray:
74      """Compute␣the␣gain␣matrix␣K␣through␣Ricatti␣recursion
75
76  ␣␣␣␣Args:
77  ␣␣␣␣␣␣␣␣A␣(np.ndarray):␣Dynamics␣matrix,␣shape␣(n,␣n)
78  ␣␣␣␣␣␣␣␣B␣(np.ndarray):␣Controls␣matrix,␣shape␣(n,␣m)
79  ␣␣␣␣␣␣␣␣Q␣(np.ndarray):␣State␣cost␣matrix,␣shape␣(n,␣n)
80  ␣␣␣␣␣␣␣␣R␣(np.ndarray):␣Control␣cost␣matrix,␣shape␣(m,␣m)
81
82  ␣␣␣␣Returns:
83  ␣␣␣␣␣␣␣␣np.ndarray:␣Gain␣matrix␣K,␣shape␣(m,␣n)
84  ␣␣␣␣"""
85      eps = 1e-4  # Riccati recursion convergence tolerance
86      max_iters = 1000  # Riccati recursion maximum number of iterations
87      P_prev = np.zeros((n, n))  # initialization
88      converged = False
89      for i in range(max_iters):
90          # PART (b) ###################################################
91          # INSTRUCTIONS: Apply the Ricatti equation until convergence
92          K = -np.linalg.solve(R + B.T @ P_prev @ B, B.T @ P_prev @ A)
93          P = Q + A.T @ P_prev @ (A + B @ K)
94          if np.max(np.abs(P - P_prev)) < eps:
95              converged = True
96              break
97          P_prev = P
98          # END PART (b) #################################################
99      if not converged:
100         raise RuntimeError("Ricatti␣recursion␣did␣not␣converge!")
101     print("K:", K)
102     return K
103
104
105 def simulate(
106     t: np.ndarray, s_ref: np.ndarray, u_ref: np.ndarray, s0: np.ndarray, K: np.ndarray
107 ) -> tuple[np.ndarray, np.ndarray]:
108     """Simulate␣the␣cartpole
109
110 ␣␣␣␣Args:
111 ␣␣␣␣␣␣␣␣t␣(np.ndarray):␣Evaluation␣times,␣shape␣(num_timesteps,)
112 ␣␣␣␣␣␣␣␣s_ref␣(np.ndarray):␣Reference␣state␣s_bar,␣evaluated␣at␣each␣time␣t.␣Shape␣(
        num_timesteps,␣n)
113 ␣␣␣␣␣␣␣␣u_ref␣(np.ndarray):␣Reference␣control␣u_bar,␣shape␣(m,)
114 ␣␣␣␣␣␣␣␣s0␣(np.ndarray):␣Initial␣state,␣shape␣(n,)
115 ␣␣␣␣␣␣␣␣K␣(np.ndarray):␣Feedback␣gain␣matrix␣(Ricatti␣recursion␣result),␣shape␣(m,␣n)
116
117 ␣␣␣␣Returns:
118 ␣␣␣␣␣␣␣␣tuple[np.ndarray,␣np.ndarray]:␣Tuple␣of:
119 ␣␣␣␣␣␣␣␣␣␣␣␣np.ndarray:␣The␣state␣history,␣shape␣(num_timesteps,␣n)
120 ␣␣␣␣␣␣␣␣␣␣␣␣np.ndarray:␣The␣control␣history,␣shape␣(num_timesteps,␣m)
121 ␣␣␣␣"""
122
123     def cartpole_wrapper(s, t, u):
124         """Helper␣function␣to␣get␣cartpole()␣into␣a␣form␣preferred␣by␣odeint,␣which␣
```

```
            expects␣t␣as␣the␣second␣arg"""
        return cartpole(s, u)

    # PART (c) ##################################################
    # INSTRUCTIONS: Complete the function to simulate the cartpole system
    # Hint: use the cartpole wrapper above with odeint
    s = np.zeros((t.size, n))
    u = np.zeros((t.size, m))
    s[0] = s0
    for k in range(t.size - 1):
        u[k] = u_ref + K @ (s[k] - s_ref[k])
        s[k + 1] = odeint(cartpole_wrapper, s[k], t[k : k + 2], args=(u[k],))[1]
    u[-1] = u[-2]
    # END PART (c) ##############################################
    return s, u


def compute_lti_matrices() -> tuple[np.ndarray, np.ndarray]:
    """Compute␣the␣linearized␣dynamics␣matrices␣A␣and␣B␣of␣the␣LTI␣system

␣␣␣␣Returns:
␣␣␣␣␣␣␣␣tuple[np.ndarray,␣np.ndarray]:␣Tuple␣of:
␣␣␣␣␣␣␣␣␣␣␣␣np.ndarray:␣The␣A␣(dynamics)␣matrix,␣shape␣(n,␣n)
␣␣␣␣␣␣␣␣␣␣␣␣np.ndarray:␣The␣B␣(controls)␣matrix,␣shape␣(n,␣m)
␣␣␣␣"""
    # PART (a) ##################################################
    # INSTRUCTIONS: Construct the A and B matrices
    a = mp * g / mc
    b = (mc + mp) * g / (mc * L)
    F = np.array(
        [
            [0.0, 0.0, 1.0, 0.0],
            [0.0, 0.0, 0.0, 1.0],
            [0.0, a,   0.0, 0.0],
            [0.0, b,   0.0, 0.0],
        ]
    )
    A = np.eye(n) + dt * F
    B = dt * np.array([[0.0], [0.0], [1.0 / mc], [1.0 / (mc * L)]])
    # END PART (a) ##############################################
    return A, B


def plot_state_and_control_history(
    s: np.ndarray, u: np.ndarray, t: np.ndarray, s_ref: np.ndarray, name: str
) -> None:
    """Helper␣function␣for␣cartpole␣visualization

␣␣␣␣Args:
␣␣␣␣␣␣␣␣s␣(np.ndarray):␣State␣history,␣shape␣(num_timesteps,␣n)
␣␣␣␣␣␣␣␣u␣(np.ndarray):␣Control␣history,␣shape␣(num_timesteps,␣m)
␣␣␣␣␣␣␣␣t␣(np.ndarray):␣Times,␣shape␣(num_timesteps,)
␣␣␣␣␣␣␣␣s_ref␣(np.ndarray):␣Reference␣state␣s_bar,␣evaluated␣at␣each␣time␣t.␣Shape␣(
    num_timesteps,␣n)
␣␣␣␣␣␣␣␣name␣(str):␣Filename␣prefix␣for␣saving␣figures
␣␣␣␣"""
    fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
    plt.subplots_adjust(wspace=0.35)
    labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
    labels_u = (r"$u(t)$",)
    for i in range(n):
        axes[i].plot(t, s[:, i])
        axes[i].plot(t, s_ref[:, i], "--")
```

```
        axes[i].set_xlabel(r"$t$")
        axes[i].set_ylabel(labels_s[i])
    for i in range(m):
        axes[n + i].plot(t, u[:, i])
        axes[n + i].set_xlabel(r"$t$")
        axes[n + i].set_ylabel(labels_u[i])
    plt.savefig(FIG_DIR / f"{name}.png", bbox_inches="tight")
    # plt.show()

    if animate:
        fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
        ani.save(FIG_DIR / f"{name}.mp4", writer="ffmpeg")
        # plt.show()


def main():
    # Part A
    A, B = compute_lti_matrices()

    # Part B
    Q = np.eye(n)  # state cost matrix
    R = np.eye(m)  # control cost matrix
    K = ricatti_recursion(A, B, Q, R)

    # Part C
    t = np.arange(0.0, 30.0, 1 / 10)
    s_ref = np.array([0.0, np.pi, 0.0, 0.0]) * np.ones((t.size, 1))
    u_ref = np.array([0.0])
    s0 = np.array([0.0, 3 * np.pi / 4, 0.0, 0.0])
    s, u = simulate(t, s_ref, u_ref, s0, K)
    plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance")

    # Part D
    # Note: t, u_ref unchanged from part c
    s_ref = np.array([reference(ti) for ti in t])
    s0 = np.array([0.0, np.pi, 0.0, 0.0])
    s, u = simulate(t, s_ref, u_ref, s0, K)
    plot_state_and_control_history(s, u, t, s_ref, "cartpole_balance_tv")


if __name__ == "__main__":
    main()
```

cartpole_swingup.py

Answers 2.4(a), 2.4(c), and 2.4(d).

```
"""
Starter code for the problem "Cart-pole swing-up".

Autonomous Systems Lab (ASL), Stanford University
"""

import time
from pathlib import Path

from animations import animate_cartpole

import jax
import jax.numpy as jnp

```

```
import matplotlib.pyplot as plt

import numpy as np

from scipy.integrate import odeint

FIG_DIR = Path(__file__).resolve().parents[1] / "latex" / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)

def linearize(f, s, u):
    """Linearize the function `f(s, u)` around `(s, u)`.

    Arguments
    ---------
    f : callable
        A nonlinear function with call signature `f(s, u)`.
    s : numpy.ndarray
        The state (1-D).
    u : numpy.ndarray
        The control input (1-D).

    Returns
    -------
    A : numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `s`.
    B : numpy.ndarray
        The Jacobian of `f` at `(s, u)`, with respect to `u`.
    """
    # WRITE YOUR CODE BELOW ###################################################
    # INSTRUCTIONS: Use JAX to compute `A` and `B` in one line.
    A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
    ###########################################################################
    return A, B


def ilqr(f, s0, s_goal, N, Q, R, QN, eps=1e-3, max_iters=1000):
    """Compute the iLQR set-point tracking solution.

    Arguments
    ---------
    f : callable
        A function describing the discrete-time dynamics, such that
        `s[k+1] = f(s[k], u[k])`.
    s0 : numpy.ndarray
        The initial state (1-D).
    s_goal : numpy.ndarray
        The goal state (1-D).
    N : int
        The time horizon of the LQR cost function.
    Q : numpy.ndarray
        The state cost matrix (2-D).
    R : numpy.ndarray
        The control cost matrix (2-D).
    QN : numpy.ndarray
        The terminal state cost matrix (2-D).
    eps : float, optional
        Termination threshold for iLQR.
    max_iters : int, optional
        Maximum number of iLQR iterations.

    Returns
    -------
    s_bar : numpy.ndarray
```

```
        A 2-D array where `s_bar[k]` is the nominal state at time step `k`,
        for `k = 0, 1, ..., N-1`
    u_bar : numpy.ndarray
        A 2-D array where `u_bar[k]` is the nominal control at time step `k`,
        for `k = 0, 1, ..., N-1`
    Y : numpy.ndarray
        A 3-D array where `Y[k]` is the matrix gain term of the iLQR control
        law at time step `k`, for `k = 0, 1, ..., N-1`
    y : numpy.ndarray
        A 2-D array where `y[k]` is the offset term of the iLQR control law
        at time step `k`, for `k = 0, 1, ..., N-1`
    """
    if max_iters <= 1:
        raise ValueError("Argument `max_iters` must be at least 1.")
    n = Q.shape[0]  # state dimension
    m = R.shape[0]  # control dimension

    # Initialize gains `Y` and offsets `y` for the policy
    Y = np.zeros((N, m, n))
    y = np.zeros((N, m))

    # Initialize the nominal trajectory `(s_bar, u_bar)`, and the
    # deviations `(ds, du)`
    u_bar = np.zeros((N, m))
    s_bar = np.zeros((N + 1, n))
    s_bar[0] = s0
    for k in range(N):
        s_bar[k + 1] = f(s_bar[k], u_bar[k])
    ds = np.zeros((N + 1, n))
    du = np.zeros((N, m))

    def cost(s, u):
        J = 0.5 * (s[N] - s_goal) @ QN @ (s[N] - s_goal)
        for k in range(N):
            J += 0.5 * (s[k] - s_goal) @ Q @ (s[k] - s_goal)
            J += 0.5 * u[k] @ R @ u[k]
        return J

    # iLQR loop
    converged = False
    for _ in range(max_iters):
        # Linearize the dynamics at each step `k` of `(s_bar, u_bar)`
        A, B = jax.vmap(linearize, in_axes=(None, 0, 0))(f, s_bar[:-1], u_bar)
        A, B = np.array(A), np.array(B)

        # PART (c) ###################################################
        # INSTRUCTIONS: Update `Y`, `y`, `ds`, `du`, `s_bar`, and `u_bar`.
        q = [Q @ (s_bar[k] - s_goal) for k in range(N)]
        r = [R @ u_bar[k] for k in range(N)]
        P = QN.copy()
        p = QN @ (s_bar[N] - s_goal)

        for k in reversed(range(N)):
            G = R + B[k].T @ P @ B[k]
            Y[k] = -np.linalg.solve(G, B[k].T @ P @ A[k])
            y[k] = -np.linalg.solve(G, r[k] + B[k].T @ p)
            p = q[k] + A[k].T @ p + A[k].T @ P @ B[k] @ y[k]
            P = Q + A[k].T @ P @ (A[k] + B[k] @ Y[k])

        ds.fill(0.0)
        du.fill(0.0)
        for k in range(N):
            du[k] = Y[k] @ ds[k] + y[k]
```

```
            ds[k + 1] = A[k] @ ds[k] + B[k] @ du[k]

        J_prev = cost(s_bar, u_bar)
        accepted = False
        for α in (1.0, 0.5, 0.25, 0.1, 0.05, 0.01):
            u_next = u_bar + α * du
            s_next = np.zeros_like(s_bar)
            s_next[0] = s0
            for k in range(N):
                s_next[k + 1] = f(s_next[k], u_next[k])
            if np.all(np.isfinite(s_next)) and cost(s_next, u_next) < J_prev:
                accepted = True
                break
        if not accepted:
            α = 0.0
            u_next = u_bar
            s_next = s_bar
            du.fill(0.0)
        u_bar = u_next
        s_bar = s_next
        ############################################################################

        if np.max(np.abs(α * du)) < eps:
            converged = True
            break
    if not converged:
        raise RuntimeError("iLQR did not converge!")
    return s_bar, u_bar, Y, y


def cartpole(s, u):
    """Compute the cart-pole state derivative."""
    mp = 2.0  # pendulum mass
    mc = 10.0  # cart mass
    L = 1.0  # pendulum length
    g = 9.81  # gravitational acceleration

    x, θ, dx, dθ = s
    sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
    h = mc + mp * (sinθ**2)
    ds = jnp.array(
        [
            dx,
            dθ,
            (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
            -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
            / (h * L),
        ]
    )
    return ds


# Define constants
n = 4  # state dimension
m = 1  # control dimension
Q = np.diag(np.array([10.0, 10.0, 2.0, 2.0]))  # state cost matrix
R = 1e-2 * np.eye(m)  # control cost matrix
QN = 1e2 * np.eye(n)  # terminal state cost matrix
s0 = np.array([0.0, 0.0, 0.0, 0.0])  # initial state
s_goal = np.array([0.0, np.pi, 0.0, 0.0])  # goal state
T = 10.0  # simulation time
dt = 0.1  # sampling time
animate = False  # flag for animation
```

```
closed_loop = False  # flag for closed-loop control

# Initialize continuous-time and discretized dynamics
f = jax.jit(cartpole)
fd = jax.jit(lambda s, u, dt=dt: s + dt * f(s, u))

# Compute the iLQR solution with the discretized dynamics
print("Computing iLQR solution ... ", end="", flush=True)
start = time.time()
t = np.arange(0.0, T, dt)
N = t.size - 1
s_bar, u_bar, Y, y = ilqr(fd, s0, s_goal, N, Q, R, QN)
print("done! ({:.2f} s)".format(time.time() - start), flush=True)

def simulate_and_plot(closed_loop):
    """Simulate either the open-loop sequence or closed-loop iLQR policy."""
    print("Simulating {} ... ".format("closed-loop" if closed_loop else "open-loop"), end=
        "", flush=True)
    start = time.time()
    s = np.zeros((N + 1, n))
    u = np.zeros((N, m))
    s[0] = s0
    for k in range(N):
        # PART (d) ###################################################################
        # INSTRUCTIONS: Compute either the closed-loop or open-loop value of
        # `u[k]`, depending on the Boolean flag `closed_loop`.
        if closed_loop:
            u[k] = u_bar[k] + Y[k] @ (s[k] - s_bar[k]) + y[k]
        else:  # do open-loop control
            u[k] = u_bar[k]
        ##############################################################################
        s[k + 1] = odeint(lambda s, t: f(s, u[k]), s[k], t[k : k + 2])[1]
    print("done! ({:.2f} s)".format(time.time() - start), flush=True)

    fig, axes = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
    plt.subplots_adjust(wspace=0.45)
    labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
    labels_u = (r"$u(t)$",)
    for i in range(n):
        axes[i].plot(t, s[:, i])
        axes[i].set_xlabel(r"$t$")
        axes[i].set_ylabel(labels_s[i])
    for i in range(m):
        axes[n + i].plot(t[:-1], u[:, i])
        axes[n + i].set_xlabel(r"$t$")
        axes[n + i].set_ylabel(labels_u[i])
    if closed_loop:
        plt.savefig(FIG_DIR / "cartpole_swingup_cl.png", bbox_inches="tight")
    else:
        plt.savefig(FIG_DIR / "cartpole_swingup_ol.png", bbox_inches="tight")
    plt.show()

    if animate:
        fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
        ani.save(FIG_DIR / "cartpole_swingup.mp4", writer="ffmpeg")
        plt.show()


simulate_and_plot(False)
simulate_and_plot(True)
```